Compêndio Técnico: Kaggriculture Strategy Architecture

Consolidação tática dos notebooks: 'Commit for Three Days', 'Six-Day Public-State Fieldbook', 'Three-Day Shop Router' e 'Farming Score: A Mathematical Approach'. A premissa central é romper a reatividade cega e adotar janelas rígidas de decisão, métricas causais em relação ao rival e proteção orçamentária contra liquidação ineficiente de insumos.

1. Janelas Rígidas de Época e Horizontes de Decisão

- Épocas de 3 Dias (72 Turnos): Como o mercado abre em ciclos de 3 dias, flutuações intermediárias são ruído estocástico. A decisão tática congela em: e(t) = 432 + 72 * floor((t - 432) / 72) para t >= 432. Turnos de avaliação: 432, 504, 576 e 648.
- Macroblocos de 6 Dias (144 Turnos): Usados para expansões estruturais de longo prazo (terreno, ajudantes e animais).

2. Sinais de Decisão Econômica e Causal

- Diferença de Caixa: Delta C = C_own - C_rival
- Índice Normalizado de Preços Prêmio: Q = (1/3) * sum(p_i / p_base_i) para MELON, MILK e WOOL.
- Diferença de Capacidade: Delta K = sum(K_rival - K_own), comparando a malha produtiva instalada.
- Roteamento de Lojas (Turno 360): Se a primeira loja for BAKERY com fertilizante abundante no mercado (<= 10232), ou PET_CAFE com capacidade rival controlada (<= 64 plantas), ativa rota de especialização agressiva.

3. Lógica dos Três Modos de Jogo

- Protect (Proteger): Ativado se Delta C >= 1000, Q <= 0.78, celeiro sem sobrecarga e Delta K <= 0. Limita vendas a lotes mínimos (1 unidade) com margens altas (>= 90%) para não degradar o preço do rival.
- Convert (Converter): Ativado se Delta C <= -1000, Q >= 0.82 e Delta K >= 0. Despeja estoque acumulado a partir de 45% da base para forçar liquidez e recompor investimentos.
- Balanced (Equilibrado): Modo padrão estável; aplica tranches graduais de vendas conforme preço de mercado.

4. Mecanismo de Budget Guard

- Previne falhas de compra por falta pontual de caixa sem precisar liquidar sementes ou fertilizantes.
- Calcula o déficit: shortfall = Custo_Planejado - Caixa_Atual.
- Liquida itens vendáveis do celeiro priorizando o maior preço unitário primeiro até cobrir exatamente a margem.

Implementação Modular Recomendada para o Agente V4:

```
# ============================================================
# ESTADO GLOBAL E CONTROLE DE EPOCA
# ============================================================
ESTADO_EPOCA = {
    "ultimo_passo_decisao": -1,
    "modo_ativo": "balanced",
    "rota_especial": 0
}

def calcular_metricas_causais(obs, jogador):
    fazendas = obs.get("farms", [])
    rival_id = 1 - jogador if len(fazendas) >= 2 else jogador
    minha = fazendas[jogador]
    rival = fazendas[rival_id]

    delta_c = float(minha.get("money", 0.0)) - float(rival.get("money", 0.0))

    precos = obs.get("market", {}).get("prices", {})
    itens_premio = ["MELON", "STRAWBERRY", "WOOL"]
       soma_q = sum(precos.get(p, get_product_base_price(p)) / max(get_product_base_price(p), 1) for p in
itens_premio)
    q = soma_q / len(itens_premio)
```

```
    k_own = sum(1 for r in minha["tiles"] for t in r if isinstance(t, dict) and t.get("kind") == "PLANT")
    k_rival = sum(1 for r in rival["tiles"] for t in r if isinstance(t, dict) and t.get("kind") == "PLANT")
    return delta_c, q, (k_rival - k_own), k_rival

def atualizar_modo_estrategico(obs, jogador):
    step = obs["step"]
    if step == 360:
        lojas = obs.get("town", {}).get("unlocked_shops", [])
        inv_mercado = obs.get("market", {}).get("inventory", {})
        primeira_loja = lojas[0] if lojas else None
        _, _, _, k_rival = calcular_metricas_causais(obs, jogador)
        if (primeira_loja == "BAKERY" and inv_mercado.get("FERTILIZER", 99999) <= 10232) or \
           (primeira_loja == "PET_CAFE" and k_rival <= 64):
            ESTADO_EPOCA["rota_especial"] = 1

    if step < 432:
        return ESTADO_EPOCA["modo_ativo"]

    epoca = 432 + 72 * ((step - 432) // 72)
    if epoca != ESTADO_EPOCA["ultimo_passo_decisao"]:
        delta_c, q, delta_k, _ = calcular_metricas_causais(obs, jogador)
        pressao_celeiro = sum(obs["private"]["shed"].values()) >= 80

        if delta_c >= 1000 and q <= 0.78 and not pressao_celeiro and delta_k <= 0:
            ESTADO_EPOCA["modo_ativo"] = "protect"
        elif delta_c <= -1000 and q >= 0.82 and delta_k >= 0:
            ESTADO_EPOCA["modo_ativo"] = "convert"
        else:
            ESTADO_EPOCA["modo_ativo"] = "balanced"
        ESTADO_EPOCA["ultimo_passo_decisao"] = epoca

    return ESTADO_EPOCA["modo_ativo"]

def aplicar_budget_guard(obs, custo_previsto):
    caixa = obs["farms"][obs["player"]].get("money", 0.0)
    deficit = custo_previsto - caixa
    if deficit <= 0:
        return []

    celeiro = obs["private"]["shed"]
    precos = obs.get("market", {}).get("prices", {})
    candidatos = [
        {"item": k, "qtd": v, "preco": precos.get(k, get_product_base_price(k))}
        for k, v in celeiro.items() if v > 0 and k not in {"FERTILIZER"}
    ]
    candidatos.sort(key=lambda x: x["preco"], reverse=True)

    ordens = []
    for c in candidatos:
        if deficit <= 0:
            break
        qtd_necessaria = int(-(-deficit // c["preco"]))
        qtd_venda = min(c["qtd"], qtd_necessaria, 5)
        ordens.append(["SELL", c["item"], qtd_venda])
        deficit -= qtd_venda * c["preco"]
    return ordens
```